

CampusConnect — Arquitetura Frontend

Visão Geral

O frontend do CampusConnect é construído com **Vite + React 19 + TypeScript**, seguindo uma arquitetura baseada em **Feature Modules** combinada com os princípios **SOLID** e as melhores práticas de desenvolvimento React moderno.

Estrutura de Diretórios

```
src/
├── app/                                # Configuração global da aplicação
│   └── providers.tsx                  # Composição de todos os providers (Router,
│                                       Tooltip, etc.)
├── features/                          # Módulos de domínio (negócio)
│   ├── feed/                         # Feature: Feed de posts e stories
│   │   ├── components/
│   │   │   ├── StoriesBar.tsx
│   │   │   ├── PostComposer.tsx
│   │   │   └── PostCard.tsx
│   │   ├── types.ts                 # Interfaces/types do domínio
│   │   └── data.ts                 # Mock data (substituível por hooks de API)
│   ├── auth/                        # Feature: Autenticação
│   │   └── components/
│   │       └── LoginForm.tsx
│   ├── groups/                      # Feature: Grupos acadêmicos
│   │   ├── components/
│   │   │   └── GroupCard.tsx
│   │   ├── types.ts
│   │   └── data.ts
│   └── news/                        # Feature: Notícias do campus
│       ├── components/
│       │   └── NewsCard.tsx
│       ├── types.ts
│       └── data.ts
├── components/                       # Componentes compartilhados
│   ├── layout/                      # Shell do layout da aplicação
│   │   ├── Layout.tsx # Layout principal com sidebars
│   │   ├── Header.tsx # Cabeçalho global
│   │   ├── sidebar-left.tsx # Navegação lateral esquerda
│   │   └── sidebar-right.tsx # Painel lateral direito
│   └── ui/                          # Componentes de UI atômicos (shadcn/ui)
```

├─ pages/	# Componentes de página (compositores finos)
│ ├─ home.tsx	
│ ├─ login.tsx	
│ ├─ news.tsx	
│ └─ groups.tsx	
├─ router/	
│ └─ index.tsx	# Definição das rotas da aplicação
├─ hooks/	# Hooks reutilizáveis
├─ lib/	
│ └─ utils.ts	# Utilitários (cn, formatters, etc.)
├─ assets/	# Imagens, fontes, SVGs
├─ App.tsx	# Componente raiz
├─ main.tsx	# Entry point – monta o React
└─ index.css	# CSS global + variáveis do tema

Princípios SOLID Aplicados

S — Single Responsibility (Responsabilidade Única)

Cada módulo tem exatamente uma razão para mudar:

- **PostCard** renderiza apenas um post — não sabe buscar dados, não sabe montar o feed.
- **StoriesBar** cuida apenas da barra de stories.
- **Layout** gerencia apenas a estrutura do shell da dashboard.
- **Header** gerencia apenas o cabeçalho.

Regra prática: se você precisa descrever um componente com "e também", é sinal de que ele faz mais de uma coisa.

O — Open/Closed (Aberto/Fechado)

Componentes são abertos para extensão via props, mas fechados para modificação direta:

- **StoriesBar** aceita `stories?: Story[]` — pode receber dados externos sem alterar sua implementação.
- **PostCard** recebe o tipo **Post** completo — adicionar campos ao tipo estende o comportamento sem modificar o card.
- Todos os componentes propagam `className` e `...props` para permitir customização pontual.

L — Liskov Substitution (Substituição de Liskov)

Qualquer componente pode ser substituído por outro que satisfaça a mesma interface:

- **GroupCard** e **NewsCard** seguem o mesmo padrão de `{ item: T }` prop — podem ser trocados ou decorados sem impactar a página.
- **LoginForm** pode ser substituído por outra implementação de formulário de login sem alterar **LoginPage**.

I — Interface Segregation (Segregação de Interfaces)

Props interfaces são pequenas e focadas no que o componente realmente usa:

```
// ☒ Focado
interface PostCardProps {
  post: Post;
}

// ☐ Evitar
interface PostCardProps {
  post: Post;
  user: User;
  onLike: () => void;
  onComment: () => void;
  onShare: () => void;
  showAuthor: boolean;
  compact: boolean;
  // ...
}
```

Quando um componente precisa de muitas props, considere dividir em subcomponentes ou usar composição.

D — Dependency Inversion (Inversão de Dependência)

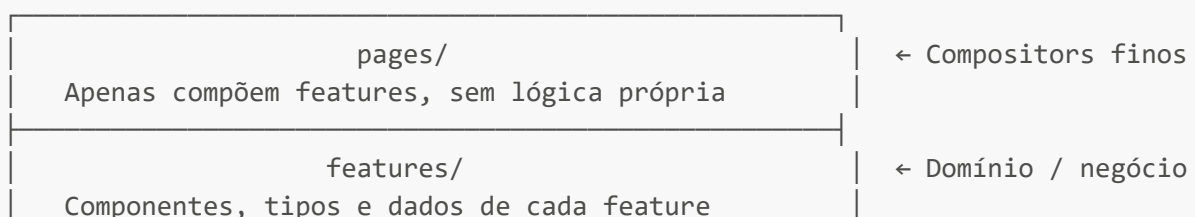
Páginas dependem de abstrações (componentes de feature), não de implementações concretas:

```
// ☒ HomePage depende da abstração StoriesBar
import { StoriesBar } from "@features/feed/components/StoriesBar";

// ☐ HomePage não deve importar diretamente dados mock ou lógica de fetch
import { mockStories } from "@features/feed/data"; // isso pertence à feature,
// não à página
```

No futuro, quando houver uma API real, apenas a camada de dados ([data.ts](#) → [hooks/useStories.ts](#)) muda — a página e os componentes permanecem intactos.

Camadas da Aplicação



components/layout/ + ui/ Shell, sidebars, primitivos shadcn	← Infraestrutura UI
hooks/ · lib/ · app/providers transversais Estado global, utilitários, configuração	← Serviços

Convenções de Nomenclatura

Tipo	Convenção	Exemplo
Componente React	PascalCase	PostCard.tsx
Hook customizado	camelCase com use	useAuth.ts
Tipo / Interface	PascalCase	interface Post {}
Arquivo de dados mock	data.ts	features/feed/data.ts
Arquivo de tipos	types.ts	features/feed/types.ts
Utilitário puro	camelCase	formatDate.ts
Página	camelCase (arquivo)	home.tsx → export HomePage

Fluxo de Dados

```

[API / Mock data]
  ↓
[feature/data.ts ou hook useXxx()]
  ↓
[feature/components/XxxComponent]
  ↓
[pages/xxx.tsx – composição]
  ↓
[router/index.tsx]
  ↓
[components/layout/Layout]
  ↓
[App.tsx → main.tsx]

```

Como Adicionar uma Nova Feature

1. Crie a pasta `src/features/<nome>/`
2. Defina os tipos em `types.ts`

3. Crie os dados mock em `data.ts` (depois substituir por hook de API)
4. Crie os componentes em `components/`
5. Crie (ou atualize) a página em `src/pages/`
6. Adicione a rota em `src/router/index.tsx`
7. Adicione o link de navegação em `src/components/layout/sidebar-left.tsx`

Stack Tecnológica

Tecnologia	Versão	Papel
React	19	UI framework
TypeScript	5.x	Tipagem estática
Vite	8.x	Build tool e dev server
Tailwind CSS	4.x	Estilização utilitária
shadcn/ui	4.x	Componentes UI acessíveis (Radix)
react-router-dom	7.x	Roteamento SPA
date-fns	4.x	Manipulação de datas